Assignment 3 - Shortest Path in an Unweighted Graph

Course: CS201 Data Structures & Algorithms
Language: Python 3
Topic: Graph traversal

Task

Implement a function that returns the shortest path between two nodes in an unweighted, directed graph given as an adjacency list:

```
def bfs_shortest_path(graph: dict[str, list[str]], start: str, goal: str) -> list[str] | None:
    ...
```

- graph maps each node id to the list of nodes reachable from it by one edge.
- Return the shortest path from start to goal as a list of node ids, **including both endpoints** (e.g. ["A", "F", "G"]). "Shortest" = fewest edges.
- If start == goal, return [start].
- If goal is unreachable from start, return None.
- The graph may contain cycles; your function must still terminate.

Guidance

In an unweighted graph the shortest path (fewest edges) is found by **breadth-first search**: explore the graph layer by layer using a FIFO queue and a visited set, so the first time you reach the goal you have used the fewest possible edges. A depth-first search will find *a* path, but not necessarily the shortest.

What we assess

1. Correctness - returns a minimal-edge path on the test graphs.
2. Algorithmic understanding - uses a breadth-first strategy (queue + visited set).
3. Edge cases - start == goal, unreachable goal (returns None), cyclic graphs.